# 会社編集ダイアログUI設計書

## メタデータ

**title: 会社編集ダイアログUI設計書**
**category: UI設計**
**version: 1.0.0**
**last_updated: 2026-01-23**
**author: Development Team**
**tags: [provider, corporation, dialog, edit]**

## 関連ドキュメント

- 会社管理メイン画面
- 会社追加ダイアログ
- マスタ管理API仕様
- 会社テーブルDDL
- 会社管理DB接続計画
- UIポリシー

---

## 2. 画面レイアウト

### 2.1. 画面イメージ




---

# 3. 概要

## 3.1. 目的

プロバイダが既存の会社情報を編集するためのダイアログを定義します。会社名とアクティブ状態を変更できます。会社IDは変更不可（主キーのため）。

## 3.2. 利用権限

- **Provider（プロバイダ）**: 使用可能

---

# 2. ダイアログ構成

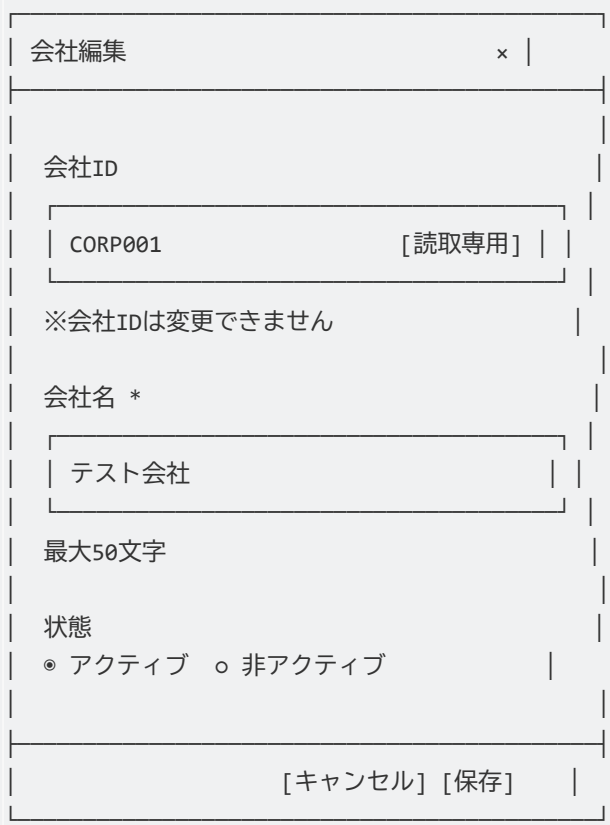
## 2.1. ダイアログ概要

**表示トリガー**: 会社管理画面の「編集」ボタンクリック

**モーダルタイプ**: Modal

**サイズ**: 中（600px）

## 2.2. レイアウト




---

# 3. 入力項目

## 3.1. 会社ID（表示のみ）

**コンポーネント**: `<Form.Item>` + `<Input>`

**仕様**:

| 項目 | 値 |
|---|---|
| ラベル | 「会社ID」 |
| 型 | text |
| 読み取り専用 | ○（disabled） |
| 説明 | 「※会社IDは変更できません」 |

**実装例**:

```
<Form.Item
  label="会社ID"
  name="corporationId"
  extra="※会社IDは変更できません"
>
  <Input disabled />
</Form.Item>
```

## 3.2. 会社名（必須）

**コンポーネント**: `<Form.Item>` + `<Input>`

**仕様**:

| 項目 | 値 |
| --- | --- |
| ラベル | 「会社名」 |
| プレースホルダー | 「会社名を入力」 |
| 型 | text |
| 必須 | ○ |
| 最小文字数 | 1文字 |
| 最大文字数 | 50文字 |

**バリデーション**:

```
{
  required: true,
  message: '会社名を入力してください'
},
{
  min: 1,
  max: 50,
  message: '会社名は1〜50文字で入力してください'
}
```

## 3.3. 状態（必須）

**コンポーネント**: `<Form.Item>` + `<Radio.Group>`

**仕様**:

| 項目 | 値 |
|---|---|
| ラベル | 「状態」 |
| 選択肢 | アクティブ / 非アクティブ |
| 必須 | ○ |

**実装例**:

```
<Form.Item
  label="状態"
  name="isActive"
  rules={[{ required: true, message: '状態を選択してください' }]}
>
  <Radio.Group>
    <Radio value={true}>アクティブ</Radio>
    <Radio value={false}>非アクティブ</Radio>
  </Radio.Group>
</Form.Item>
```

---

# 4. アクションボタン

## 4.1. キャンセルボタン

**コンポーネント**: `<Button>`

**仕様**:

| 項目 | 値 |
|---|---|
| ラベル | 「キャンセル」 |
| タイプ | `default` |
| 配置 | フッター左側 |

**動作**:

1. 入力内容を破棄
2. ダイアログを閉じる

3. 親画面に戻る

## 4.2. 保存ボタン

**コンポーネント**: `<Button>`

**仕様**:

| 項目 | 値 |
|---|---|
| ラベル | 「保存」 |
| タイプ | `primary` |
| 配置 | フッター右側 |
| ローディング | API呼び出し中は `true` |
| 無効化 | バリデーションエラー時 |

**動作**:

1. フォームバリデーション実行
2. 変更があるか確認
3. API呼び出し: `PUT /api/v1/provider/corporation/:corporationId`
4. 成功時:
    - 成功メッセージ表示（Toast）
    - ダイアログを閉じる
    - 親画面の一覧を再取得
5. 失敗時: エラーメッセージ表示

---

# 5. データの初期化

## 5.1. ダイアログ表示時

編集対象の会社データを取得してフォームに設定:

```
useEffect(() => {
  if (open && corporationId) {
    fetchCorporationData(corporationId);
  }
}, [open, corporationId]);

const fetchCorporationData = async (id: string) => {
  try {
    const response = await apiClient.get(
      `/api/v1/provider/corporation/${id}`
    );

    if (response.data.success) {
      form.setFieldsValue({
        corporationId: response.data.data.corporationId,
        corporationName: response.data.data.corporationName,
        isActive: response.data.data.isActive
      });
    }
  } catch (error) {
    message.error('会社情報の取得に失敗しました');
  }
};
```

---

# 6. APIリクエスト仕様

## 6.1. データ取得エンドポイント

`GET /api/v1/provider/corporation/:corporationId`

**リクエストヘッダー**:

```
Authorization: Bearer {jwt_token}
```

**レスポンス（成功時）**:

```
{
  "success": true,
```

```
  "data": {
    "corporationId": "CORP001",
    "corporationName": "テスト会社",
    "isActive": true,
    "createAt": "2026-01-20T10:30:00.000Z",
    "updateAt": "2026-01-23T10:30:00.000Z"
  },
  "meta": {
    "timestamp": "2026-01-23T10:30:00.000Z",
    "requestId": "req_abc123"
  }
}
```

## 6.2. 更新エンドポイント

`PUT /api/v1/provider/corporation/:corporationId`

**リクエストヘッダー**:

```
Content-Type: application/json
Authorization: Bearer {jwt_token}
```

**リクエストボディ**:

```
{
  "corporationName": "テスト会社株式会社",
  "isActive": true
}
```

**レスポンス（成功時）**:

```
{
  "success": true,
  "data": {
    "corporationId": "CORP001",
    "corporationName": "テスト会社株式会社",
    "isActive": true,
    "createAt": "2026-01-20T10:30:00.000Z",
    "updateAt": "2026-01-23T10:35:00.000Z"
  },
```

```
  "meta": {
    "timestamp": "2026-01-23T10:35:00.000Z",
    "requestId": "req_abc456"
  }
}
```

**レスポンス（エラー時）**:

```
{
  "success": false,
  "error": {
    "code": "CORPORATION_NOT_FOUND",
    "message": "会社が見つかりません",
    "details": []
  },
  "meta": {
    "timestamp": "2026-01-23T10:35:00.000Z",
    "requestId": "req_abc456"
  }
}
```

---

# 7. エラーメッセージ

## 7.1. バリデーションエラー

| エラー条件 | メッセージ |
|---|---|
| 会社名が空 | 「会社名を入力してください」 |
| 会社名が長すぎる | 「会社名は1〜50文字で入力してください」 |
| 状態が未選択 | 「状態を選択してください」 |

## 7.2. APIエラー

| エラーコード | メッセージ |
|---|---|
| `CORPORATION_NOT_FOUND` | 「会社が見つかりません」 |
| `VALIDATION_ERROR` | 「入力内容に誤りがあります」 |

| | |
|---|---|
| `UNAUTHORIZED` | 「この操作を実行する権限がありません」 |
| `INTERNAL_SERVER_ERROR` | 「サーバーエラーが発生しました。しばらく経ってから再試行してください」 |

---

# 8. 実装例（TypeScript + Next.js + Ant Design）

```tsx
// frontend/src/components/Provider/CorporationEditDialog.tsx
'use client';

import React, { useState, useEffect } from 'react';
import { Modal, Form, Input, Radio, Button, message } from 'antd';
import apiClient from '@/utils/apiClient';

interface CorporationEditDialogProps {
  open: boolean;
  onClose: () => void;
  onSuccess: () => void;
  corporationId: string | null;
}

interface CorporationFormValues {
  corporationId: string;
  corporationName: string;
  isActive: boolean;
}

export default function CorporationEditDialog({
  open,
  onClose,
  onSuccess,
  corporationId
}: CorporationEditDialogProps) {
  const [form] = Form.useForm();
  const [isLoading, setIsLoading] = useState(false);
  const [isFetching, setIsFetching] = useState(false);

  // データ取得
  useEffect(() => {
    if (open && corporationId) {
      fetchCorporationData(corporationId);
    }
  }, [open, corporationId]);

  const fetchCorporationData = async (id: string) => {
    setIsFetching(true);
```

```
  try {
    const response = await apiClient.get(
      `/api/v1/provider/corporation/${id}`
    );

    if (response.data.success) {
      const data = response.data.data;
      form.setFieldsValue({
        corporationId: data.corporationId,
        corporationName: data.corporationName,
        isActive: data.isActive
      });
    }
  } catch (error: any) {
    const errorMessage =
      error.response?.data?.error?.message || '会社情報の取得に失敗しました';
    message.error(errorMessage);
    onClose();
  } finally {
    setIsFetching(false);
  }
};

const handleSubmit = async (values: CorporationFormValues) => {
  if (!corporationId) return;

  setIsLoading(true);
  try {
    const response = await apiClient.put(
      `/api/v1/provider/corporation/${corporationId}`,
      {
        corporationName: values.corporationName,
        isActive: values.isActive
      }
    );

    if (response.data.success) {
      message.success('会社を更新しました');
      form.resetFields();
      onSuccess();
      onClose();
    }
  } catch (error: any) {
    const errorMessage =
      error.response?.data?.error?.message || '会社の更新に失敗しました';
    message.error(errorMessage);
  } finally {
    setIsLoading(false);
  }
};
```

```
const handleCancel = () => {
  form.resetFields();
  onClose();
};

return (
  <Modal
    title="会社編集"
    open={open}
    onCancel={handleCancel}
    footer={null}
    width={600}
    destroyOnClose
  >
    <Form
      form={form}
      layout="vertical"
      onFinish={handleSubmit}
    >
      {/* 会社ID（読み取り専用） */}
      <Form.Item
        label="会社ID"
        name="corporationId"
        extra="※会社IDは変更できません"
      >
        <Input disabled />
      </Form.Item>

      {/* 会社名 */}
      <Form.Item
        label="会社名"
        name="corporationName"
        rules={[
          { required: true, message: '会社名を入力してください' },
          { min: 1, max: 50, message: '会社名は1〜50文字で入力してください' }
        ]}
        extra="最大50文字"
      >
        <Input placeholder="会社名を入力" maxLength={50} />
      </Form.Item>

      {/* 状態 */}
      <Form.Item
        label="状態"
        name="isActive"
        rules={[{ required: true, message: '状態を選択してください' }]}
      >
        <Radio.Group>
          <Radio value={true}>アクティブ</Radio>
```

```
            <Radio value={false}>非アクティブ</Radio>
          </Radio.Group>
        </Form.Item>

        {/* フッター */}
        <Form.Item style={{ marginBottom: 0, marginTop: 24 }}>
          <div style={{ display: 'flex', justifyContent: 'flex-end', gap: 8 }}>
            <Button onClick={handleCancel}>キャンセル</Button>
            <Button
              type="primary"
              htmlType="submit"
              loading={isLoading || isFetching}
              disabled={isLoading || isFetching}
            >
              保存
            </Button>
          </div>
        </Form.Item>
      </Form>
    </Modal>
  );
}
```

---

# 9. 変更検知と最適化

## 9.1. 変更なし判定

フォームの値が初期値と同じ場合、APIを呼び出さずに確認メッセージを表示:

```
const handleSubmit = async (values: CorporationFormValues) => {
  // 初期値と比較
  const initialValues = form.getFieldsValue();
  const hasChanged =
    values.corporationName !== initialValues.corporationName ||
    values.isActive !== initialValues.isActive;

  if (!hasChanged) {
    message.info('変更がありません');
    return;
  }
```

```
  // API呼び出し...
};
```

## 9.2. 確認ダイアログ

重要な変更（非アクティブ化）の場合、確認ダイアログを表示:

```
import { Modal } from 'antd';

const handleSubmit = async (values: CorporationFormValues) => {
  // 非アクティブ化の場合は確認
  if (!values.isActive && initialValues.isActive) {
    Modal.confirm({
      title: '会社を非アクティブ化しますか？',
      content: '非アクティブにすると、この会社を使用できなくなります。',
      okText: 'はい',
      cancelText: 'いいえ',
      onOk: () => submitUpdate(values)
    });
  } else {
    submitUpdate(values);
  }
};
```

---

# 10. テスト要件

## 10.1. 単体テスト

### テストケース

- データ取得機能のテスト
- バリデーション機能のテスト
- フォーム送信処理のテスト
- 変更検知機能のテスト

## 10.2. 統合テスト

### テストシナリオ

1. 正常な会社編集
2. 会社名のみ変更
3. 状態のみ変更
4. 変更なしで保存
5. 存在しない会社の編集
6. 非アクティブ化の確認ダイアログ

---

# 11. アクセシビリティ

## 11.1. キーボード操作

- Tab: 次の入力項目へフォーカス移動
- Shift + Tab: 前の入力項目へフォーカス移動
- Enter: フォーム送信
- Escape: ダイアログを閉じる

## 11.2. スクリーンリーダー対応

- 読み取り専用フィールドに `aria-readonly="true"` を設定
- エラーメッセージを `role="alert"` で通知
- 必須項目を `aria-required="true"` で明示

---

# 12. 変更履歴

| バージョン | 日付 | 変更内容 | 担当者 |
|---|---|---|---|
| 1.0.0 | 2026-01-23 | 初版作成 | Development Team |